Object Oriented Programming

Classes, Inheritance, Encapsulation & More

Shwetank's Python Journey • OOP Module • 10 Core Concepts

The Four Pillars of OOP

1	**Encapsulation** — Bundling data and methods together; hiding internal details
2	**Inheritance** — A class can acquire properties and methods from another class
3	**Polymorphism** — Same method name, different behaviour depending on the class
4	**Abstraction** — Exposing only necessary details; hiding complex implementation

NOTE	All 10 problems below explore these pillars using one running example: a Car and ElectricCar class. Watch how they build on each other.

① Basic Class and Object — *class + __init__*

A class is a blueprint. An object is an instance created from that blueprint. __init__ is the constructor — it runs automatically when you create a new object and sets up its attributes.

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand    # instance attribute
        self.model = model    # instance attribute

# Creating instances (objects) from the class
my_car    = Car('Toyota', 'Camry')
your_car  = Car('Honda',  'Civic')

print(my_car.brand)   # Toyota
print(your_car.model) # Civic
```

▶ **Output**

```
Toyota
Civic
```

NOTE self refers to the specific object being created or used. When you write self.brand = brand, you're storing brand on that particular instance, not shared across all cars.

? Interview Q: What is the difference between a class and an object?

☑ A class is the template/blueprint (like an architectural plan). An object is a concrete instance built from that template (like an actual house). You can create many objects from one class, each with its own data.

② Class Method and Self *— instance methods*

Methods are functions defined inside a class. Every instance method takes self as its first parameter — Python passes the object itself automatically so the method can access the object's data.

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def full_name(self):            # instance method
        return f'{self.brand} {self.model}'

    def describe(self):
        return f'This is a {self.full_name()}'  # can call other methods

my_car = Car('Toyota', 'Camry')
print(my_car.full_name())  # Toyota Camry
print(my_car.describe())   # This is a Toyota Camry
```

▶ Output

```
Toyota Camry
This is a Toyota Camry
```

TIP my_car.full_name() is Python shorthand for Car.full_name(my_car). Python inserts the object as the first argument automatically — that's what self IS.

③ Inheritance *— parent → child class*

Inheritance lets a child class reuse all the attributes and methods of a parent class and add its own extras. super() calls the parent's __init__ so you don't have to repeat it.

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def full_name(self):
        return f'{self.brand} {self.model}'


class ElectricCar(Car):            # ElectricCar inherits from Car
    def __init__(self, brand, model, battery_size):
        super().__init__(brand, model)  # run Car's __init__
        self.battery_size = battery_size  # extra attribute


my_tesla = ElectricCar('Tesla', 'Model S', 100)
print(my_tesla.full_name())      # inherited method still works!
print(my_tesla.battery_size)     # 100
```

▶ **Output**

```
Tesla Model S
100
```

NOTE	ElectricCar gets full_name() for free — it inherits it from Car. This is the DRY (Don't Repeat Yourself) principle in action: define once, reuse everywhere.

? Interview Q: What does super() do?

☑ super() returns a proxy to the parent class. super().__init__(...) calls the parent's constructor, so the child doesn't have to copy-paste all that setup code. It also makes the code safer if the inheritance chain changes.

④ Encapsulation — *private attributes + getters*

Encapsulation means hiding an object's internal data and only exposing it through controlled methods. In Python, prefix an attribute with __ (double underscore) to make it private — Python will name-mangle it so it can't be accessed directly from outside.

```
class Car:
    def __init__(self, brand, model):
        self.__brand = brand   # __ makes it 'private'
```

```
        self.model   = model

    def get_brand(self):       # getter method
        return self.__brand

    def set_brand(self, brand):  # setter method (optional)
        if isinstance(brand, str) and brand:
            self.__brand = brand

my_car = Car('Toyota', 'Camry')
print(my_car.get_brand())    # Toyota ☑
# print(my_car.__brand)      # AttributeError ✕  — name-mangled
# print(my_car._Car__brand)  # works but bad practice — don't do this
```

▶ **Output**

```
Toyota
```

> ⚠ **WARN** Python's 'private' is a convention enforced by name mangling, not a hard lock. __brand becomes _Car__brand internally. Other languages (Java, C++) enforce true private access; Python trusts the developer.

? Interview Q: What is name mangling in Python?

☑ When you write self.__brand, Python internally renames it to _Car__brand. This prevents accidental access from outside and from subclasses. It's a convention-based protection, not a true access modifier like in Java.

⑤ Polymorphism *— same method, different behaviour*

Polymorphism means 'many forms'. Both Car and ElectricCar define a fuel_type() method, but each returns a different answer. Code that calls .fuel_type() doesn't need to know which type it's dealing with — it just calls the method.

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def fuel_type(self):
        return 'Petrol / Diesel'


class ElectricCar(Car):
    def __init__(self, brand, model, battery_size):
```

```
        super().__init__(brand, model)
        self.battery_size = battery_size

    def fuel_type(self):          # overrides parent method
        return 'Electric Charge'


cars = [Car('Honda', 'City'), ElectricCar('Tesla', 'Model 3', 82)]

for car in cars:                  # same method call, different result
    print(f'{car.brand}: {car.fuel_type()}')
```

▶ **Output**

```
Honda: Petrol / Diesel
Tesla: Electric Charge
```

NOTE This is method overriding — the child class provides its own version of a method inherited from the parent. The for loop demonstrates polymorphism: the same .fuel_type() call produces different results based on the actual object type.

⑥ Class Variables — *shared across all instances*

A class variable belongs to the class itself, not to any one instance. It's shared across all objects of that class. Useful for counters, constants, or defaults that all instances should share.

```
class Car:
    total_cars = 0              # class variable — shared by ALL
instances

    def __init__(self, brand, model):
        self.brand = brand      # instance variable — unique per object
        self.model = model
        Car.total_cars += 1     # update class variable via class name

car1 = Car('Toyota', 'Camry')
car2 = Car('Honda',  'Civic')
car3 = Car('Tesla',  'Model 3')

print(Car.total_cars)    # 3  — accessed on the CLASS
print(car1.total_cars)   # 3  — also accessible on instances
```

▶ **Output**

```
3
3
```

> ⚠ **WARN** Always update a class variable using the class name (Car.total_cars), not self.total_cars. Writing self.total_cars += 1 would create a new instance variable on that object, hiding the class variable rather than updating it.

? Interview Q: What is the difference between a class variable and an instance variable?

☑ A class variable is defined in the class body (outside __init__) and shared across all instances — one copy for all. An instance variable is defined with self.x inside __init__ — each object gets its own independent copy.

⑦ Static Method — *@staticmethod*

A static method belongs to the class namespace but doesn't receive self or cls — it doesn't need access to the instance or class. Use it for utility functions that are logically related to the class but don't depend on any object's state.

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    @staticmethod
    def general_description():     # no self, no cls
        return 'Cars are four-wheeled motor vehicles used for
transportation.'

    @staticmethod
    def is_valid_speed(speed):     # utility — no instance needed
        return 0 <= speed <= 300

# Call on the class (no object needed)
print(Car.general_description())
print(Car.is_valid_speed(120))     # True
print(Car.is_valid_speed(500))     # False

# Can also be called on an instance (works but unusual)
my_car = Car('Toyota', 'Camry')
print(my_car.general_description())
```

▶ Output

```
Cars are four-wheeled motor vehicles used for transportation.
True
False
Cars are four-wheeled motor vehicles used for transportation.
```

Static Method	Class Method (bonus)

`@staticmethod`
`No self or cls`
`Use: utility/helper logic`
`Call: Class.method()`
	`@classmethod`
	`Receives cls (the class)`
	`Use: alternative constructors`
	`Call: Class.method()`

⑧ Property Decorators — *@property — read-only attributes*

The @property decorator turns a method into a read-only attribute. It lets you access it like a plain attribute (no parentheses), while controlling what happens when someone tries to set or delete it.

```
class Car:
    def __init__(self, brand, model):
        self.brand  = brand
        self.__model = model   # private storage

    @property
    def model(self):           # getter — accessed as car.model
        return self.__model

    # NO @model.setter defined → model is read-only

my_car = Car('Toyota', 'Camry')
print(my_car.model)        # Camry ☑  (no parentheses needed)

# my_car.model = 'Corolla' # AttributeError ✕  — can't set it
```

▶ **Output**

```
Camry
```

TIP	Add @model.setter to allow writes with validation. Add @model.deleter for controlled deletion. Properties give you Java-style getters/setters but with clean Python attribute syntax.

? Interview Q: Why use @property instead of a regular get_model() method?

☑ @property lets you write car.model (clean attribute syntax) instead of car.get_model() (Java-style method call). It also lets you add validation or computation later without changing how external code accesses the attribute.

⑨ isinstance() and Inheritance Checking

isinstance(obj, Class) returns True if obj is an instance of Class or any of its parent classes. This lets you check types while respecting the inheritance hierarchy.

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

class ElectricCar(Car):
    def __init__(self, brand, model, battery_size):
        super().__init__(brand, model)
        self.battery_size = battery_size


my_tesla = ElectricCar('Tesla', 'Model S', 100)
my_honda = Car('Honda', 'City')

print(isinstance(my_tesla, ElectricCar))  # True  — direct class
print(isinstance(my_tesla, Car))           # True  — parent class!
print(isinstance(my_honda, ElectricCar))  # False — not an ElectricCar
print(isinstance(my_honda, Car))           # True

# type() for exact match only
print(type(my_tesla) == Car)               # False
print(type(my_tesla) == ElectricCar)       # True
```

▶ **Output**

```
True
True
False
True
False
True
```

NOTE	isinstance(my_tesla, Car) is True because ElectricCar inherits from Car — a Tesla IS-A Car. This IS-A relationship is the heart of inheritance. type() only checks the exact class, not the hierarchy.

? Interview Q: When would you use isinstance() in real code?

☑ When writing functions that need to behave differently based on type — e.g., a garage function that calls battery_status() only on ElectricCars. Prefer isinstance() over type() because it respects subclasses.

⑩ Multiple Inheritance *— MRO — Method Resolution Order*

Python allows a class to inherit from more than one parent class simultaneously. When methods with the same name exist in multiple parents, Python uses the MRO (Method Resolution Order) — always left-to-right — to decide which one to use.

```
class Battery:
    def __init__(self, battery_size=75):
        self.battery_size = battery_size

    def battery_info(self):
        return f'Battery: {self.battery_size} kWh'


class Engine:
    def __init__(self, horsepower=150):
        self.horsepower = horsepower

    def engine_info(self):
        return f'Engine: {self.horsepower} HP'


class ElectricCar(Battery, Engine):   # multiple inheritance
    def __init__(self, brand, battery_size, horsepower):
        Battery.__init__(self, battery_size)   # explicit parent init
        Engine.__init__(self, horsepower)
        self.brand = brand

    def full_info(self):
        return f'{self.brand} | {self.battery_info()} |
{self.engine_info()}'


my_tesla = ElectricCar('Tesla', 100, 670)
print(my_tesla.full_info())
print(ElectricCar.__mro__)   # see the resolution order
```

▶ **Output**

```
Tesla | Battery: 100 kWh | Engine: 670 HP
(<class '__main__.ElectricCar'>, <class '__main__.Battery'>, <class
'__main__.Engine'>, <class 'object'>)
```

NOTE __mro__ shows exactly the order Python searches for methods. For ElectricCar: first ElectricCar itself, then Battery, then Engine, then the base object class. This prevents the 'diamond problem' found in other languages.

⚠ WARN	Multiple inheritance can get complex fast. The preferred Python pattern is using Mixins — small focused classes that add one specific ability (like Battery, Engine above). Keep each mixin focused on one responsibility.

? Interview Q: What is the Diamond Problem and how does Python solve it?

☑ When D inherits from B and C, which both inherit from A, calling D.method() is ambiguous — should it use B's or C's version? Python's MRO (C3 Linearisation algorithm) defines a deterministic left-to-right order, so there's never ambiguity.

OOP Quick Reference

Concept	Syntax	Key Point
Class / Object	`class Car:  /  Car()`	Blueprint vs instance
__init__	`def __init__(self, ...):`	Constructor — runs on creation
self	`self.brand = brand`	Reference to the current object
Inheritance	`class E(Car):`	Child gets all parent methods
super()	`super().__init__(...)`	Calls parent constructor
Encapsulation	`self.__brand`	__ makes it 'private' (name-mangled)
Polymorphism	`def fuel_type(self):`	Same name, different behaviour
Class Variable	`total = 0  (outside init)`	Shared by all instances
@staticmethod	`@staticmethod`	No self/cls — utility function
@property	`@property`	Read-only attribute access
isinstance()	`isinstance(obj, Class)`	Checks type + parent classes
Multiple inherit.	`class E(Battery, Engine):`	Left-to-right MRO

OOP is just code organised around real-world things. A Car is a great place to start — and a chai machine would be even better! 🥥